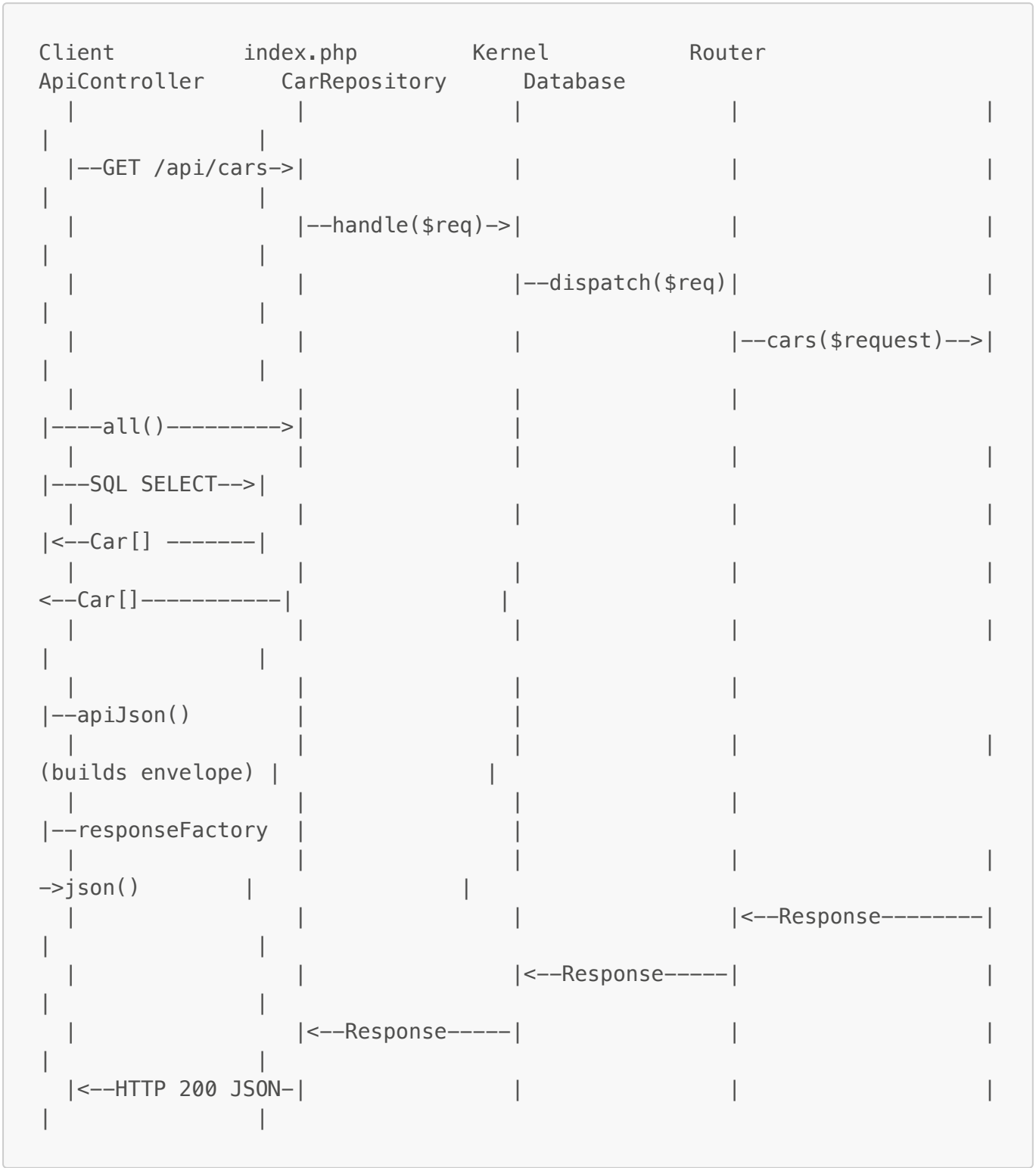
Answers — REST Routes in an Existing Backend

Teacher-facing document. Not for distribution to students.

Exercise 1 — Sequence Diagram for `GET /api/cars`

Model sequence diagram



Answers to follow-up questions

1. At which point is the JSON envelope assembled?

Inside `ApiBaseController::apiJson()`, called from `ApiController::cars()`. It builds the `['meta' => ..., 'links' => ..., 'data' => ...]` array before passing it to `ResponseFactory::json()`.

2. Where is it decided that the response body should be JSON and not HTML?

In `ResponseFactory::json()` — it sets `$response->header = "Content-Type: application/json"` and encodes the body with `json_encode()`. The decision is made by the controller calling `apiJson()` rather than `view()`.

3. If `CarRepository::all()` returns an empty array, what does the response look like?

```
{
  "meta":  { "resource": "cars", "total": 0 },
  "links": { "self": "/api/cars" },
  "data":  []
}
```

Status code is still 200 — an empty collection is a valid, successful response. Returning 404 for an empty list is a common mistake (404 means the resource itself doesn't exist, not that it has no items).

Grading note

- **Weak:** diagram includes only 3–4 participants; return arrows missing; envelope assembly not identified
- **Adequate:** all participants present; correct flow direction; can identify where JSON is built
- **Strong:** correct return types on arrows (e.g., `Car[]`, `Response`); all three follow-up questions answered with precise references to the code

---

Exercise 2 — Add the Makes API Routes

Model implementation

`app/Controllers/ApiController.php` — additions:

```
use App\Models\Make;
use App\Repositories\MakeRepositoryInterface;

// In constructor: add MakeRepositoryInterface $makeRepository parameter
// and store as $this->makeRepository

public function makes(Request $request): Response
{
    $makes = $this->makeRepository->all();
    $data = array_map([$this, 'makeToArray'], $makes);
    return $this->apiJson('makes', $data, $request->path, total:
```

```
count($data));
}

public function make(Request $request): Response
{
    $id = (int)$request->get('id');
    $make = $this->makeRepository->find($id);
    if ($make === null) {
        return $this->apiError('makes', 'Make not found', $request->path,
404, '/api/makes');
    }
    return $this->apiJson('makes', $this->makeToArray($make), $request-
>path, '/api/makes');
}

/** @return array<string, mixed> */
private function makeToArray(Make $make): array
{
    return [
        'id'          => $make->id,
        'name'        => $make->name,
        'country'     => $make->country,
        'description' => $make->description,
    ];
}
```

`app/RouteProvider.php` — additions:

```
use App\Controllers\MakeApiController; // not needed — stays in
ApiController

$apiController = $container->get(ApiController::class);
// add alongside existing api routes:
$router->addRoute('GET', '/api/makes', [$apiController, 'makes']);
$router->addRoute('GET', '/api/makes/(?<id>\d+)', [$apiController,
'make']);
```

Expected responses

`GET /api/makes`:

```
{
  "meta":  { "resource": "makes", "total": 5 },
  "links": { "self": "/api/makes" },
  "data":  [ { "id": 1, "name": "Ford", "country": "USA", "description":
"..." }, "..." ]
}
```

`GET /api/makes/1`:

```
{
  "meta":  { "resource": "makes" },
  "links": { "self": "/api/makes/1", "collection": "/api/makes" },
  "data":  { "id": 1, "name": "Ford", "country": "USA", "description":
"..." }
}
```

`GET /api/makes/999`:

```
{
  "meta":  { "resource": "makes" },
  "links": { "self": "/api/makes/999", "collection": "/api/makes" },
  "error": { "status": 404, "message": "Make not found" }
}
```

Extension: nested cars

Students need to add a `findByMake(int $makeId): array` (or similar) method to `CarRepository`, then include the result in `makeToArray()`:

```
'cars' => array_map(
    fn($car) => ['id' => $car->id, 'model' => $car->model, 'year' => $car-
>year],
    $this->carRepository->findByMake($make->id)
),
```

Grading note

- **Weak:** routes registered but controller not wired (DI error); or response shape does not match the envelope
- **Adequate:** both routes work; 404 case handled; envelope shape correct
- **Strong:** constructor injection done cleanly; `makeToArray()` extracted as a private method; 404 includes `collection` link; extension implemented

---

Exercise 3 — Add the Categories API Routes

Model implementation

Same pattern as Makes. Key differences:

- Resource name: `'categories'`
- Fields: `id`, `name` only
- Collection link: `/api/categories`

```
public function categories(Request $request): Response
{
    $categories = $this->categoryRepository->all();
    $data = array_map([$this, 'categoryToArray'], $categories);
    return $this->apiJson('categories', $data, $request->path, total:
count($data));
}

public function category(Request $request): Response
{
    $id = (int)$request->get('id');
    $category = $this->categoryRepository->find($id);
    if ($category === null) {
        return $this->apiError('categories', 'Category not found',
$request->path, 404, '/api/categories');
    }
    return $this->apiJson('categories', $this->categoryToArray($category),
$request->path, '/api/categories');
}

private function categoryToArray(Category $category): array
{
    return ['id' => $category->id, 'name' => $category->name];
}
```

Routes:

```
$router->addRoute('GET', '/api/categories', [$apiController,
'categories']);
$router->addRoute('GET', '/api/categories/(?<id>\d+)', [$apiController,
'category']);
```

Grading note

Identical criteria to Exercise 2. Since this is homework following the same pattern, the bar is higher: a strong response needs no structural guidance from the teacher.

Exercise 4 — Reflect and Argue

Key elements a strong answer includes

- **Takes a position.** Either "yes, it is RESTful" or "it depends / technically yes but..."
- **Identifies that query parameters are acceptable in REST.** `/api/cars/search?q=mustang` is not wrong per se — query parameters are a standard way to filter a collection resource.
- **Distinguishes between the URL naming concern and the REST constraint.** `/search` as a sub-path risks looking like a verb/action; a cleaner design might be `GET /api/cars?q=mustang`, keeping `/api/cars` as the resource and letting the query string handle filtering.

- **Mentions idempotence or safety.** A `GET` with a query parameter is still safe and idempotent — this is fine.
- **Acknowledges the trade-off.** Filtering via query strings (`?q=`) is conventional; a separate `/search` path is common in practice (GitHub, Elasticsearch) even if purists object.

What distinguishes weak / adequate / strong

- **Weak:** "yes it's fine" or "no it's not RESTful" with no reasoning
- **Adequate:** identifies the resource vs. action distinction; suggests `?q=` alternative
- **Strong:** references a specific REST constraint by name; acknowledges the pragmatic counter-argument; states a clear recommendation with a reason